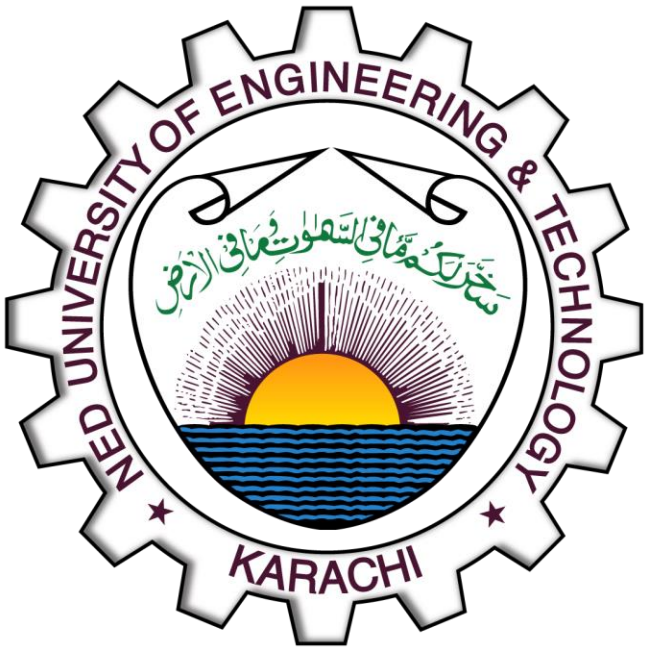
DATA STRUCTURE ALGORITHMS & APPLICATIONS

[CT-159]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: SSCS-D

LAB : #06

QUESTION#1:

CODE:

```cpp
#include <iostream>
using namespace std;

void bubbleSort(int *arr, size_t n)
{
    for (size_t i = 0; i < n - 1; i++)
    {
        cout << "PASS " << i + 1 << endl;
        bool swapped = false;
        for (size_t j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
            {
                swapped = true;
                swap(arr[j], arr[j + 1]);
            }
        if (!swapped)
            return;
    }
}

void print(int *arr, size_t n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int arr[] = {5, 1, 4, 8};
    size_t N = sizeof(arr) / sizeof(arr[0]);
    cout << "Given array: ";
    print(arr, N);
    bubbleSort(arr, N);
    cout << "Sorted array: ";
    print(arr, N);
    return 0;
}
```

OUTPUT:

```
Given array: 5 1 4 8
PASS 1
PASS 2
Sorted array: 1 4 5 8
```

QUESTION#2:

CODE:

```cpp
#include <iostream>
using namespace std;

void sort(int *arr, int n)
{
    for (size_t i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        for (; j >=0 && arr[j] > key; j--)
            arr[j + 1] = arr[j];
        arr[j + 1] = key;
    }
}

size_t maxToys(int *arr, size_t N, int K) {
    sort(arr, N);
    size_t count = 0;
    for (size_t i = 0; i < N; i++)
        if (arr[i] <= K) {
            K-= arr[i];
            count++;
        } else break;
    return count;
}

int main() {
    size_t N, K;
    cout << "Enter N (len): ";
    cin >> N;
    cout << "Enter K (amt): ";
    cin >> K;
    int *arr = new int[N];
    cout << "Enter Toys amt: ";
    for (int i = 0; i < N; i++)
        cin >> arr[i];
    cout << "Number of toys you can buy: " << maxToys(arr, N, K) << endl;
    delete[] arr;
    return 0;
}
```

OUTPUT:

```
Enter N (len): 7
Enter K (amt): 50
Enter Toys amt: 1 12 5 111 200 1000 10
Number of toys you can buy: 4
```

QUESTION#3:

CODE:

```
#include <bits/stdc++.h>
#include <ctime>
#include <cstdlib>
using namespace std;

struct SortMetrics
{
    size_t comparisons = 0, swaps = 0;
};

class Sort
{
private:
    static void bubble(vector<int> &arr, SortMetrics &metrics)
    {
        for (size_t i = 0; i < arr.size() - 1; i++)
            for (size_t j = 0; j < arr.size() - i - 1; j++)
            {
                metrics.comparisons++;
                if (arr[j] > arr[j + 1])
                {
                    metrics.swaps++;
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
    }

    static void insertion(vector<int> &arr, SortMetrics &metrics)
    {
        for (size_t i = 1; i < arr.size(); i++)
        {
            int key = arr[i];
            int j = static_cast<int>(i) - 1;
            while (j >= 0)
            {
                metrics.comparisons++;
                if (arr[j] > key)
                {
```

```
                    arr[j + 1] = arr[j];
                    metrics.swaps++;
                }
                j--;
            }
            arr[j + 1] = key;
        }
    }

    static void selection(vector<int> &arr, SortMetrics &metrics)
    {
        for (size_t i = 0; i < arr.size() - 1; i++)
        {
            size_t min = i;
            for (size_t j = i + 1; j < arr.size(); j++)
            {
                metrics.comparisons++;
                if (arr[j] < arr[min])
                    min = j;
            }
            if (min != i)
            {
                metrics.swaps++;
                int temp = arr[i];
                arr[i] = arr[min];
                arr[min] = temp;
            }
        }
    }

    static void printRow(const string& name, size_t comps, size_t swaps, size_t
total) {
        cout << left << setw(15) << name
                  << setw(18) << comps
                  << setw(15) << swaps
                  << total << "\n";
    }
public:
    static void sort(vector<int> &arr, const string &type)
    {
        SortMetrics dummy;
        if (type == "bubble") {
            bubble(arr, dummy);
        } else if (type == "insertion") {
            insertion(arr, dummy);
```

```
        } else if (type == "selection") {
            selection(arr, dummy);
        } else {
            cout << "\e[1;31mUnkown sorting technique: " << type << "\n\e[0m";
        }
    }

    static void analyze(size_t size) {
        vector<int> original(size);
        for (int i = 0; i < size; i++)
            original[i] = rand() % 1000 + 1; // random number: 1-1000

        vector<int> bArr = original;
        vector<int> iArr = original;
        vector<int> sArr = original;

        SortMetrics bMetrics, iMetrics, sMetrics;

        bubble(bArr, bMetrics);
        insertion(iArr, iMetrics);
        selection(sArr, sMetrics);

        cout << "\n============================================================\n";
        cout << "  SORTING ANALYSIS (N = " << size << " Random Elements)\n";
        cout << "============================================================\n";
        cout << left << setw(15) << "Algorithm"
                  << setw(18) << "Comparisons"
                  << setw(15) << "Swaps/Shifts"
                  << "Total Ops\n";
        cout << "------------------------------------------------------------\n";

        size_t bTotal = bMetrics.comparisons + bMetrics.swaps;
        size_t iTotal = iMetrics.comparisons + iMetrics.swaps;
        size_t sTotal = sMetrics.comparisons + sMetrics.swaps;

        printRow("Bubble", bMetrics.comparisons, bMetrics.swaps, bTotal);
        printRow("Insertion", iMetrics.comparisons, iMetrics.swaps, iTotal);
        printRow("Selection", sMetrics.comparisons, sMetrics.swaps, sTotal);

        cout << "------------------------------------------------------------\n";

        // determine best and worst based on overall operations
        vector<pair<string, size_t>> results = {
            {"Bubble Sort", bTotal},
            {"Insertion Sort", iTotal},
```

```
            {"Selection Sort", sTotal}
        };

        auto best = min_element(results.begin(), results.end(),
            [](const auto& a, const auto& b) { return a.second < b.second; });
        auto worst = max_element(results.begin(), results.end(),
            [](const auto& a, const auto& b) { return a.second < b.second; });

        cout << "\033[1;32m[BEST TECHNIQUE]\033[0m  : " << best->first
                  << " (" << best->second << " total operations)\n";
        cout << "\033[1;31m[WORST TECHNIQUE]\033[0m : " << worst->first
                  << " (" << worst->second << " total operations)\n";
        cout << "=========================================================\n";
    }
};

void print(const vector<int> &arr)
{
    for (auto element : arr)
        cout << element << " ";
    cout << endl;
}

int main()
{
    srand(time(0)); // random number generator seed
    cout << "Enter elements (enter 0 to stop):\n";
    vector<int> arr;
    int input;
    while (true)
    {
        cin >> input;
        if (input)
            arr.push_back(input);
        else
            break;
    }
    cout << "Enter choice of sorting:\n";
    cout << "1. bubble\n";
    cout << "2. insertion\n";
    cout << "3. selection\n";
    string choice;
    cin >> choice;
    cout << "Entered array: ";
    print(arr);
```

```
    cout << "----------------------\nSorting using: " << choice << endl;

    Sort::sort(arr, choice);
    cout << "Sorted Array: ";
    print(arr);

    size_t size;
    cout << "Enter size of analysis: ";
    cin >> size;
    Sort::analyze(size);
    return 0;
}
```

OUTPUT:

```
Enter elements (enter 0 to stop):
8
2
4
6
2
1
0
Enter choice of sorting:
1. bubble
2. insertion
3. selection
insertion
Entered array: 8 2 4 6 2 1
----------------------
Sorting using: insertion
Sorted Array: 1 2 6 8 8 8
Enter size of analysis: 50

=======================================================
  SORTING ANALYSIS (N = 50 Random Elements)
=======================================================
Algorithm      Comparisons       Swaps/Shifts   Total Ops
-------------------------------------------------------
Bubble         1225              683            1908
Insertion      1225              1049           2274
Selection      1225              47             1272
-------------------------------------------------------
[BEST TECHNIQUE]  : Selection Sort (1272 total operations)
[WORST TECHNIQUE] : Insertion Sort (2274 total operations)
=======================================================
```

QUESTION#4:

CODE:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<int> pancakeSort(vector<int> &arr)
{
    vector<int> result;
    int n = arr.size();
    for (int size = n; size > 1; --size)
    {
        int maxIdx = 0;
        for (int i = 1; i < size; i++)
            if (arr[i] > arr[maxIdx])
                maxIdx = i;

        if (maxIdx == size - 1)
            continue;

        if (maxIdx != 0) {
            reverse(arr.begin(), arr.begin() + maxIdx + 1);
            result.push_back(maxIdx + 1);
        }

        reverse(arr.begin(), arr.begin() + size);
        result.push_back(size);
    }
    return result;
}

void print(const vector<int> &arr)
{
    for (auto element : arr)
        cout << element << " ";
    cout << endl;
}

int main()
{
    cout << "Enter elements (enter 0 to stop):\n";
```

```
    vector<int> arr;
    int input;
    while (true)
    {
        cin >> input;
        if (input)
            arr.push_back(input);
        else
            break;
    }
    cout << "Entered array: ";
    print(arr);
    vector<int> flipsIdx = pancakeSort(arr);
    cout << "Sorted array: ";
    print(arr);
    cout << "Flips (k): ";
    print(flipsIdx);
    return 0;
}
```

OUTPUT:

```
Enter elements (enter 0 to stop):
3
2
4
1
0
Entered array: 3 2 4 1
Sorted array: 1 2 3 4
Flips (k): 3 4 2 3 2
```

QUESTION#5:

CODE:

```
#include <iostream>
#include <vector>
using namespace std;

void sortColors(vector<int>& colors) {
    // given that the array only has elements as 0, 1, 2
    int low = 0, mid = 0, high = colors.size() - 1;
    while (mid <= high)
    {
        if (colors[mid] == 0)
            swap(colors[low++], colors[mid++]);
        else if (colors[mid] == 1)
            mid++;
        else
            swap(colors[mid], colors[high--]);
    }
}

void print(const vector<int> &arr)
{
    for (auto element : arr)
        cout << element << " ";
    cout << endl;
}

int main()
{
    cout << "Enter elements (0-2) (enter -1 to stop):\n";
    vector<int> arr;
    int input;
    while (true)
    {
        cin >> input;
        if (input > 2 || input < 0)
            break;
        else
            arr.push_back(input);
    }
    cout << "Entered array: ";
    print(arr);
```

```
    sortColors(arr);
    cout << "After Sorting: ";
    print(arr);
    return 0;
}
```

OUTPUT:

```
Enter elements (0-2) (enter -1 to stop):
2
0
2
1
1
0
-1
Entered array: 2 0 2 1 1 0
After Sorting: 0 0 1 1 2 2
```